“Design and Analysis of Optimized Multi-Mode Taps FIR Filters”

Major Project Report

Submitted in Partial Fulfillment of the

Requirements for the Degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

By

Aditya Khuttan

(22BEC053)

Under the Guidance of

Dr. Ankur Changela



Department of Electronics and Communication Engineering,

School of Technology, Pandit Deendayal Energy University,

Gandhinagar 382426

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Project entitled "Design and Analysis of Optimized Multi-Mode Taps FIR Filters" submitted by me for the partial fulfillment of the degree of Bachelor of Technology to the Dept. of Electronics and Communication Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me under the supervision of Prof. Ankur Changela.

I also declare that this written submission adheres to University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Aditya Khuttan

Roll Number of the Student: 22BEC053

Signature of the Student:

Name of the Supervisor: Dr. Ankur Changela

Designation of the Supervisor: Assistant Professor

Signature of the Supervisor:

Place:

Date:

Certificate from the Project Supervisor/Head

I hereby declare that the B.Tech. Project entitled "Design and Analysis of Optimized Multi-Mode Taps FIR Filters" submitted by Mr. Aditya Khuttan, Roll No. 22BEC053 towards the partial fulfilment of the requirements for the award of degree in Bachelor of Technology in the field of Electronics and Communication Engineering from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is the record of work carried out by him under my supervision and guidance. The work submitted by the student has in my opinion reached a level required for being accepted for examination. The results embodied in this major project work to the best of our knowledge have not been submitted to any other University or Institution for the award of any degree or diploma.

Name and Sign of the Supervisors
(Dr. Ankur Changela)

Name and Sign of the HoD
(Dr. Parth Thakar)

Name and Sign of the Director
(Dr. Anirbid Sircar)

Place:

Date:

Acknowledgement

I am very grateful to my advisor **Dr. Ankur Changela** for his excellent guidance, constant support, and input which we found very useful throughout the course of this B.Tech thesis. His knowledge and support played a large role in the success of this work and in my growth as a professional in the field. Also I would like to thank the examiners for their in depth evaluation, for which we are very grateful, also for the productive feedback and great advice which improved the quality of this thesis.

I would like to thank the **Department of Electronics and Communication Engineering** at Pandit Deendayal Energy University for providing the resources and setting a great environment which enabled the successful completion of this project.

Aditya Khuttan

Abstract

FIR filters are the essential building blocks of digital signal processing because of their stability, linear phase property, and ability to implement them in hardware. As the need to have energy-efficient and high performance systems continue to rise, the design of low-power FIR filters has become very important especially in FPGA-based systems. This project introduces the architecture and coding of the low-power FIR filters in Verilog HDL and simulated on Xilinx Vivado, and verified by the analysis in MATLAB. This work is aimed at designing FIR filters with various tap lengths (4-10 taps) and to compare their performance of non-optimized and optimized structures. The non-optimized designs use traditional multiply-accumulate designs and the optimized designs aim at minimizing computational complexity and maximizing efficiency using methods such as symmetry of coefficients and optimization of resources. Verilog code and testbench are written and simulated to verify proper functionality and performance of every filter configuration. The designed filters are discussed in the context of power consumption, propagation delay, and hardware resource usage, such as DSP blocks, LUTs, and registers. The findings show that the optimized FIR filter designs are much better than the non-optimized ones. A maximum power savings of about 4.44% and delay savings of up to 45.75% are obtained. Also, optimized designs have better hardware utilization, which means that they require less area. These gains are even greater as the number of taps increases, proving the scalability of the optimized strategy. The frequency response of the filters are also analyzed using MATLAB to ensure that the planned systems are suitable to achieve the intended signal processing specifications. The applicability of the proposed designs in real-time scenarios is confirmed by FPGA implementation. On the whole, this paper illustrates an effective design of FIR filters, balancing power, speed and area, and is applicable to recent VLSI and digital signal processing applications.

INDEX

Chapter No.	Title	Page No.
	Acknowledgement	**iv**
	Abstract	**v**
	Index	**vi**
	List of Figures	**ix**
	List of Tables	**x**
	Nomenclature	**xi**
1	**Introduction**	1
	1.1 Background	1
	1.2 Motivation	2
	1.3 Objective	3
	1.4 Problem Statement	4
	1.5 Scope of the Project	5
	1.6 Organization of the Rest of the Report	6
2	**Literature Review**	7
	2.1 Introduction	7
	2.2 Previous FIR Filters Architectures work	8
	2.3 Low-Power FIR Filter Work Approach	9
	2.4 Faster Speed and Better Optimized FIR Filter Approach	10
	2.5 MAC and Multipliers Techniques	11
	2.6 Implementation of FPGA-Based FIR Filters	12
	2.7 Comparative Analysis of the Existing Work	13
	2.8 Research Gaps	14
	2.9 Motivation and Summary of the Proposed Work	15

3 Methodologies and Approaches used 17

3.1 FIR Filter Design 17

3.2 Non-Optimized FIR Filter Architecture 19

3.3 Optimized FIR Filter Architecture 20

3.4 Verilog Implementation 25

3.5 Designing of Testbench 25

3.6 Simulation Using Xilinx Vivado 26

3.7 MATLAB Analysis 26

3.8 FPGA Implementation 27

3.9 Performance Parameters 28

3.10 Summary 28

4 Implementation strategy 29

4.1 FIR Filter Design Flow 29

4.2 MATLAB-Based Coefficient Generation 31

4.3 Verilog HDL Implementation 34

4.4 Implementation of Non-Optimized FIR Filter 35

4.5 Optimized FIR Filter Implementation 36

4.6 Verification of Functional Logic and Testbench Design 37

4.7 RTL Implementation and Hardware FPGA Synthesis 38

4.8 Summary 40

5	**Results and Discussion**		41
	5.1	Results of MATLAB Simulation	41
	5.2	Simulations Using Xilinx Vivado	44
	5.3	RTL Synthesis Results	48
	5.4	Hardware Implementation and FPGA Resource Utilization Analysis	48
	5.5	Comparison of Non-Optimized and Optimized Architectures	50
	5.6	Filter Performance Analysis	58
	5.7	Result Discussions	60
	5.8	Summary	62
6	**Conclusions and Future Scope**		63
	6.1	Concluding Remarks	63
	6.2	Major Contributions of the Work	64
	6.3	Extensions of the Proposed Work	65
	References		69

LIST OF FIGURES

Figure No.	Title	Page No.
2.6	FPGA Nexys A7-100T Board	12
3.7	MATLAB Plots for LPF, HPF, BPF, BSF	27
4.1	Flow Chart of FIR Filter Design Flow	30
4.2.1	4-10 Taps LPF Coefficients	32
4.2.2	4-10 Taps HPF Coefficients	32
4.2.3	4-10 Taps BPF Coefficients	33
4.2.4	4-10 Taps BSF Coefficients	33
4.4	Non-Optimized Verilog Code for 10-Tap	35
4.5	Optimized Verilog Code for 10-Tap	36
4.6	Testbench for 10-Tap without and with optimized	37
4.7	RTL Analysis for 10-Tap without and with optimized and XDC Constraint File for 4-Tap for without and with optimized	39
5.1	Graph Plots of Magnitude vs Frequency for LPF, HPF, BPF, BSF	43
5.2.1	Without and With Optimized Simulation of 4-Tap FIR	44
5.2.2	Without and With Optimized Simulation of 5-Tap FIR	45
5.2.3	Without and With Optimized Simulation of 6-Tap FIR	45
5.2.4	Without and With Optimized Simulation of 7-Tap FIR	46
5.2.5	Without and With Optimized Simulation of 8-Tap FIR	46
5.2.6	Without and With Optimized Simulation of 9-Tap FIR	47
5.2.7	Without and With Optimized Simulation of 10-Tap FIR	47
5.4	FPGA Implementation of 4-Tap without and with optimized	49
5.5.1	Area Analysis of 10-Tap for both non-optimized and optimized	51
5.5.2	Delay Analysis of 10-Tap for both non-optimized and optimized	52
5.5.3	Power Analysis of 10-Tap for both non-optimized and optimized	53
5.7	Results and Comparison of Power, Delay, Area, Throughput vs Taps	61

LIST OF TABLES

Table No.	Title	Page No.
2.7	Comparison of the Previous Work from the References	13
5.5	Power, Delay, Area Analysis for 4-10 Taps without and with optimized and Improvement of Power, Delay and Total LUT	56
5.6	Various Filters Performance	59

NOMENCLATURE

Abbreviations

FIR	Finite Impulse Response
MAC	Multiply and Accumulate
HDL	Hardware Description Language
FPGA	Field Programmable Gate Array
DSP	Digital Signal Processing
ce	Clock Enable
IoT	Internet of Things
LPF	Low Pass Filter
HPF	High Pass Filter
BPF	Band Pass Filter
BSF	Band Stop Filter
FDA	Filter Designer App
CSD	Canonical Signed Digit
RCA	Ripple Carry Adder
LUT	Look Up Table
RTL	Register Transfer Level
Reg	Registers
WO	Without Optimized
OPT	Optimized
Imp	Improvement

Chapter 1

Introduction

1.1 Background

FIR filters are one of the most widely used digital filters in the signal processing due to their inherent stability, linear phase characteristics, and ease of implementation in hardware. These filters are extensively applied in areas such as communication systems, audio processing, biomedical signal processing, and image processing. With the rapid growth of portable and real-time systems, there is an increasing demand for high-speed and low-power digital signal processing architectures. As a result, efficient implementation of FIR filters on hardware platforms such as FPGAs has become a vital area of research.

FPGAs are the preferred choice for implementing FIR filters because they offer great flexibility, scalability, parallel capability, and support for real-time operation. Using the Xilinx Vivado design environment and Verilog HDL, FIR filters can be designed with efficient implementation of filter architectures for many high-performance applications in [1][2].

Conventional FIR filter architectures require many multipliers and adders thus increasing the complexity of hardware implementation, increasing the propagation delay, and increasing the power consumed. To offer an efficient FPGA FIR filter implementation several optimization techniques have been proposed, including the exploitation of coefficient symmetry, pipelining, resource sharing, and low-power arithmetic structures in [6][7][14].

The proposed work will investigate the implementation of optimized FIR filter architectures on FPGAs designed with Verilog HDL and Xilinx Vivado for use in 4-tap to 10-tap configurations. Various optimization techniques, including the exploitation of coefficient symmetry, low power operation using ce, pipelining, and DSP will be used to enhance timing performance and decrease hardware utilization.

1.2 Motivation

The growing requirement of low-power and high-performance systems in recent applications like wireless communication, embedded systems, and IoT devices has increased the need of optimized digital filter architectures. FIR filters are computationally expensive and need to be designed with efficient design techniques to minimize power consumption, delay, and achieve accuracy.

Several researchers have looked at improving FIR filter performance [6][9][14] using optimization techniques such as pipelining, coefficient symmetry exploitation, multiplier optimization and MAC optimization techniques. However, the majority of the current literature primarily discusses singular optimization techniques rather than combinations of techniques for optimization.

The motivation for this work is to create scalable, optimized architectures for FIR filters that incorporate multiple optimization techniques for increasing:

1. FPGA resource utilization,
2. Timing performance,
3. Power consumption,
4. Hardware efficiency

It also investigates the filtering performance of even-tap and odd-tap FIR filter architectures for LPF, HPF, BPF and BSF implementations.

Moreover, FPGA implementations are flexible and scalable, and they are applicable in real-time applications. But, when poorly designed, it can result in overuse of resources and higher power usage. Hence, it is vital to design a powerful FIR filter with a combination of power, delay, and area. This is the impetus behind the current work, which aims at creating and analyzing optimized FIR filter architectures in Verilog HDL and FPGA tools.

1.3 Objective

This project's aim is to design and implement low power FIR filters which will range in tap length and to study their performance in both optimized and non-optimized architectures.

This proposed work has several key objectives:

1. To create FIR filter architectures with 4, 5, 6, 7, 8, 9, and 10 taps.
2. To create coefficient values for each of the FIR filters using MATLAB FDATool.
3. To use Verilog HDL and Xilinx Vivado to build the FIR filter architectures.
4. To take advantage of coefficient symmetry, pipelining, ce low-power operation, resource sharing, and DSP/logic optimization to develop optimized FIR filter architectures.
5. To evaluate the relative performance of both the optimized and un-optimized FIR filter architectures based on their use of FPGA resources, timing performance, and power consumption.
6. To analyze the impacts of using even-tap vs. odd-tap FIR filter architectures for LPF, HPF, BPF, and BSF applications.

1.4 Problem Statement

FIR filters see great deal of arithmetic operations which in turn mostly of multiplications and additions that cause high power consumption and increased hardware complexity. In traditional implementations this brings about higher delay and poor use of FPGA resources. We set out to design a better FIR filter architecture which reduces power consumption, minimizes the delay and at the same time improves resource use without trade off performance.

The design of more complex hardware, as well as timing considerations, for higher-order FIR filters causes significant challenges for real-time DSP applications via efficient FPGA implementations.

Numerous methods have been developed to optimize FIR filter architectures however most of these attempts to do so are based on individually optimizing multipliers, pipelining, or low power arithmetic elements. Examples of previous work can be found in [3][6][9] etc.

In very limited instances, there has been publications that have combined optimization such as:

1. exploiting coefficient symmetry,
2. pipelining,
3. using ce to allow low power,
4. sharing resources,
5. using a balanced adder tree for DSP optimization

This implies that there is still a need for creating FPGA based scalable and/or optimized FIR filters which would improve timing performance, decrease hardware requirement, and decrease power consumption.

1.5 Scope of the Project

This project reports on the design and development of FIR filters in Verilog HDL and their simulation in Xilinx Vivado. We look at both which architectures we have optimized and which are not for various tap lengths. Also we use MATLAB for analysis of filter features like frequency response. We look at a scope which is limited to FPGA based implementation and performance evaluation in terms of power, delay and resource use. In this work,we looked at advanced techniques like that of coefficient optimization and efficient use of multipliers to improve and get performance.

1.6 Organization of the Report

This thesis has been structured into six chapters which comprise the following:

- The First chapter provides an introduction to the research problem being addressed, including motivation for pursuing this research, the specific objectives of the research, the research problem statement, and the scope and boundaries of the research.
- The Second chapter provides a review of the literature about FIR filter optimization techniques and FPGA implementation techniques, as well as what has been previously published on similar subject matter.
- The Third chapter discusses the methods proposed to optimize the FIR filter architectures being designed, as well as which optimization techniques will be used to design the FIR filter architecture.
- The Fourth chapter discusses how to design the FIR filter architecture and implement it using Verilog HDL in Xilinx Vivado.
- The Fifth chapter discusses the results obtained from simulating the FIR filter architectures (both optimized and non-optimized), as well as the synthesis reports, timing analysis, power analysis, and performance comparisons between optimized and non-optimized architectures resulting from the simulation of both architectures.
- The Sixth chapter discusses conclusions derived from the research, and makes suggestions about future research directions.

Chapter 2

Literature Review

2.1 Introduction

FIR filters are a valuable input filter for digital signal processing because of their high stability, linear phase response, and easy implementation. FIR filters are also commonly used in communication, medical signal processing, audio processing, image processing, and embedded systems utilizing FPGAs.

Since the demand for high-speed and low-power digital systems has increased, research on the optimal implementation of FIR filters has become significant. Several optimization techniques have been proposed to improve the performance of FIR filters in terms of hardware utilization, timing performance, throughput, and power consumption.

FIR filter architectures have been optimized by various means, such as coefficient symmetry, pipelining, multiplier optimization, resource sharing, low-power arithmetic, and MAC optimization. FPGA-based implementation with Verilog HDL and Xilinx Vivado has also been attractive due to flexibility, scalability, and real-time processing.

This chapter reviews the published research on the implementation and optimization of FIR filters on FPGAs. Methods to implement, improve, and hardware-efficient FIR filter implementation are reviewed in depth.

2.2 Previous FIR Filters Architectures work

A number of researchers have proposed different FIR filter architectures for implementing effective digital signal processing. Akram and Sheikh [11] designed and implemented a delay line filter on a tap using high-speed digital signal processing. They optimized the hardware and signal processing performance through the delay line. Daitx et al. [1] have proposed generating optimized FIR filters via VHDL for use in FPGAs and focused on increasing the flexibility of the FPGA hardware and FPGA implementation by automatically creating FIR filters. Dwivedi and Dwivedi [2] implemented FIR filters in MATLAB FDATool and Xilinx Vivado. Coefficient generation in MATLAB and FPGA realization in Verilog HDL were reported. Rao et al. [18] developed a MAC-based FIR filter for low-power applications. They found that their architecture was arithmetically efficient and utilized less FPGA power.Singh and Prakash [20] investigated higher-order FIR filters on FPGAs and demonstrated scalability and cost-effective FPGA realizations for signal processing applications.These results suggest that the FPGA-based FIR filters provide flexibility, scalability, and efficiency in digital signal processing.

2.3 Low-Power FIR Filter Work Approach

FPGA-based embedded systems and portable electronic devices have seen steady growth in their need for low-power FIR filter implementations. According to Maheswari et al. [3], an energy-efficient asynchronous 8×8 multiplier architecture for low-power digital systems performs as a low-power digital system by reducing switching activity and improving power efficiency. In the research carried out by Sumalatha et al. [5], a Vedic multiplier implementation of a low-power, low-area FIR filter architecture for the purpose of denoising ECG signals is performed. Their research indicates that this architecture reduces the device's hardware complexity and improves the efficiency of the FPGA.

An input-conscious approximate MAC unit proposed by Masadeh et al. [9] provides energy-efficient arithmetic operations by reducing power consumption while keeping an acceptable accuracy of computation. Lyakhov et al. [10] discussed the advantages of using a truncated MAC architecture in building a FIR filter to improve and lessen the arithmetic overhead associated with the performance of digital filters. Rashidi et al. [19] implemented low-power FIR filter architectures on Xilinx FPGA platforms utilizing optimized multipliers and adders. Their designs exhibit a reduction of power supply demand and an increase in hardware efficiency. These examples prove that the application of low-power various optimization techniques to FIR filters allows for a significantly reduced consumption of power by FPGAs while still permitting stable filtering to occur at the same level of performance as compared to a higher-power-filtered device.

2.4 Faster Speed and Better Optimized FIR Filter Approach

Many optimization methods aimed at enhancing timing efficiency, throughput, and multiplication efficiency in FIR filters have been developed. Kumar et al. [14] have demonstrated that by taking advantage of coefficient symmetry in traditional polyphase FIR filters, they have been able to lower the number of multipliers required, thus increasing the efficiency of the hardware through the use of symmetric coefficient configurations. Khoo et al. [7] have shown that by creating FIR filters with CSD coefficients, they have significantly reduced the amount of arithmetic required and the number of multipliers needed.Li et al. [8] produced a polynomial time FIR filter design algorithm using powers of two coefficients. The advantages of this approach generated lighter arithmetic workloads and improved efficiency when implementing FIR filter designs in hardware.Patali and Kassim [6] have presented high throughput architectures for FIR filter designs through the use of retiming and two-level pipelined techniques. These architectures exhibited lower propagation delays and higher operational frequencies. Lastly, Kishore and Raj [17] provided high throughput FIR filter architectures through the utilization of parallel processing techniques for the effective implementation of higher order filters.

The conclusion from these research efforts shows that the use of optimization techniques greatly improves a firm's implementation efficiency and temporal performance and efficiency regarding arithmetic operations when designing FIR filter architectures on an FPGA.

2.5 MAC and Multipliers Techniques

The hardware complexity and power consumption of FIR filter architectures are greatly influenced by the use of multiplicand and MAC units in FIR filters. Therefore, Multiplier and MAC optimization techniques are a key part of the successful implementation of FIR filters in an FPGA. Sakthimohan and Deny [4] developed a 16-tap FIR filter architecture by using Radix-4 Booth Multipliers and improved Booth Recoding algorithms. Their implementation provided an improvement in speed performance and reduction of the multiplication complexity. Akter and Mazumdar [12] created an FIR filter architecture using Braun Multipliers and RCA. This design improved the arithmetic efficiency of the FIR filter while also enhancing the speed performance of the FIR filter in an FPGA. George et al. [16] built their efficient FIR filter architecture through the development of double MAC Units; this allowed for an increase in the throughput and arithmetic performance of the FIR filter in an FPGA.Rao et al. [18] shared an implementation of an FIR filter architecture based on MAC operations designed specifically for low-power applications. Their architecture allowed for efficient arithmetic operations and minimal implementation of an FPGA. The above studies show that by employing Multiplier and MAC optimization techniques, there can be a significant increase in the arithmetic efficiency of the FIR filter as well as an increase in the overall performance of the FIR filter.

2.6 Implementation of FPGA-Based FIR Filters

Flexible, scalable, and high-speed realization of digital FIR filters can be achieved through the use of FPGAs. Several researchers have explored the use of FPGAs and HDLs in implementing FIR Filter architectures. Ashim and Mathur [13] conducted an implementation of a high-speed 8-tap FIR Filter using FPGA technology for use in a signal processing application. Their architecture demonstrated greater throughput along with increased FPGA performance. Dwivedi and Dwivedi [2] developed FIR Filters using the MATLAB FDATool and Xilinx Vivado in order to realize the FIR filters with an FPGA. Daitx, et al. [1] proposed a method for generating optimized FIR Filters utilizing VHDL-based implementation methods to improve FPGA resource usage and flexibility of design. Singh and Prakash [20] developed higher-order FIR Filter architectures on FPGAs and provided evidence of the scalability of DSP applications. The performance of FIR filters can be greatly enhanced by using an FPGA to accelerate their implementation via dedicated hardware datatypes. The combined execution of multiple filtering operations at once provides significant savings on computational resources. Optimized MATLAB and Xilinx Vivado implementations of FIR filters based on FPGAs have also been cited in previous literature [2][20].These five publications support the conclusion that FPGAs are an efficient means to realize scalable FIR Filter architectures that meet the needs of Real-Time DSP applications.



Figure 2.6: FPGA Nexys A7-100T Board

2.7 Comparative Analysis of the Existing Work

Table 2.7 has a comparison of the FIR filter implementations done by other researchers.

Table 2.7: Comparison of the Previous Work from the References

References	Techniques Used	Advantages	Drawbacks
[1]	Coefficient symmetry	Reduced multipliers	Complex scaling
[3]	Low-power multiplier	Reduced switching activity	Limited throughput
[5]	Vedic multiplier	Low power and area	Increased design complexity
[6]	Pipelining and retiming	High speed	Increased register count
[7]	CSD coefficients	Reduced arithmetic complexity	Coefficient dependency
[9]	Approximate MAC	Reduced power consumption	Accuracy reduction
[10]	Truncated MAC	Improved performance	Approximation error
[17]	Parallel processing	High throughput	Increased hardware utilization
[19]	Low-power FPGA FIR	Reduced power	Limited scalability
[20]	Higher-order FPGA FIR	Scalable architecture	Increased complexity

The researchers studied their implementations mainly through various optimization techniques, with one at a time being optimized using either the pipelined, multiplier optimized, and low-power arithmetic approaches separately.

2.8 Research Gaps

The literature review has highlighted a number of optimization approaches available for FIR filter implementations. A few of these examples can be found within the works that have focused primarily on single optimization methodologies as opposed to the incorporation and juxtaposition of multiple optimization methods. Few studies have been performed to document the combined use of:

1. Coefficient Symmetry Exploitation
2. Pipelining
3. ce-based, Low Power Implementation
4. Resource Sharing
5. Balanced Adder Tree Structures
6. DSP Optimization

In addition, few studies have performed comparative analyses to examine the difference between four-tap and ten-tap FIR filter architectures using optimized and non-optimized FIR filter architectures. Furthermore, very limited studies exist in the literature that compare the performance characteristics of even-tap versus odd-tap FIR filter architectures; this difference is especially important since many LPF, HPF, BPF, & BSF filter types have been implemented as either even-tap or odd-tap FIR filters. Thus, additional studies are required to create a scalable, optimized FPGA-based FIR filter architecture that will develop results regarding their respective hardware efficiency, timing performance, power consumption, and filtering characteristics while utilizing a combined approach to optimization of the FIR filter architecture.

2.9 Motivation and Summary of the Proposed Work

The growing need for low power/high-performance, FPGA-based DSPs is driving efforts to create new FIR filter architectures through the proposed method.

Some of the main intents of this work are:

1. To utilize FPGA resources less,
2. To reduce the amount of power used,
3. To increase the amount of time it takes,
4. To develop FIR filters that can be scaled,
5. To efficiently implement these FIR filters on FPGAs by applying optimization methods and techniques.

The proposed project's focus will be to implement and analyze the performance of optimized FIR filter architectures ranging from 4 taps to 10 taps in size using Verilog HDL and Xilinx Vivado software. The existing chapter has presented a thorough examination of the literature pertaining to both previous work in FIR filter optimization and FPGA implementation techniques, including the following optimization techniques such as coefficient symmetry exploitation, pipelining, low-power arithmetic structures, multiplier optimization, and FPGA-based implementation methods.

From the literature review there were gaps identified regarding both combined optimization techniques and scalable implementations of FIR filter architectures in FPGA; therefore, the proposed work will focus on the implementation and analysis of optimized FIR filter architectures with configurations between 4 taps to 10 taps implemented in FPGA technology.

Chapter 3

Methodologies and Approaches used

This chapter gives the methodologies and approaches that were adopted in the design and implementation of low-power FIR filters. This work is focused to develop non-optimized and optimized FIR filter architecture and to analyze their performance in terms of power consumption, delay, and utilization of hardware resources. The design is introduced with the help of Verilog HDL, simulated in Xilinx Vivado and analyzed using MATLAB.

3.1 Design of Multi-Mode FIR Filter

The proposed work is based on multi-mode FIR filter architecture which can be used as LPF, HPF, BPF, and BSF. Multi-mode filter implementation enhances the flexibility of the hardware and facilitates using a single architecture for multiple filtering functions without the need to use multiple hardware modules. The proposed architecture achieves different filter responses by only changing the filter mode which changes the FIR filter coefficients. The FIR coefficients for LPF, HPF, BPF and BSF are created with MATLAB FDATool and are stored in the Verilog HDL implementation. The MODE SELECT input controls which set of coefficients to use for filter operation.

The FIR filter is a digital filter, the output of which is determined solely by the current and past input samples. The overall filter equation of an FIR filter can be expressed as:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where h[k] defines the filter coefficients associated with the selected mode of the filter, x[n-k] is the input signal and y[n] is the output signal. In this project, FIR filters of various tap lengths between 4-10 taps are designed to study the variations in performance with higher order.

In the mode selection logic, a 2-bit mode selection control signal is used for selecting the required mode of the filter:

1. 00 → LPF
2. 01 → HPF
3. 10 → BPF
4. 11 → BSF

Filter coefficients are chosen depending on the desired frequency response, and are confirmed with MATLAB. These coefficients are then coded in hardware with Verilog HDL.FIR filters work really well with FPGAs because they behave in a predictable way and can process things in parallel, which makes them very efficient [8][14].

Efficient re-use of same hardware architecture for multiple filter responses is possible through coefficient switching. There had been similar attempts reported in previous research works to enhance the hardware utilization efficiency and scalability of the multi-functional FIR filter implementation by using FPGA [2][13][20].

The multi-mode architecture proposed reduces the overall hardware redundancies and optimizes the utilization of the FPGA resources by not implementing individual filter structures separately.

3.2 Non-Optimized FIR Filter Architecture

The non-optimized FIR filter is realized with the help of the traditional MAC architecture. In this structure, every input sample is multiplied by the respective coefficient and all the products are added to produce the output.

The architecture contains: shift registers, multipliers, adders and output registers.

The implementation process includes:

1. Using shift registers for delay,
2. Multiplication with coefficients,
3. Accumulation of products,
4. Output generation

It is simple, straightforward and yet leads to higher power consumption and increased hardware utilization because of the utilization of multiple multipliers and adders. There is also increased delay in the architecture by the long critical path.

The more taps the more multipliers, adders, the higher power consumption, the more DSP use, and the greater propagation delay.

The non-optimized architecture has been implemented separately for different values of taps for FIR filters (4 tap, 5 tap, 6 tap, 7 tap, 8 tap, 9 tap, and 10 tap) for comparison with the optimized architecture.

3.3 Optimized FIR Filter Architecture

To enhance the performance of the FIR filter, optimized architecture is adopted by minimizing computational complexity and maximizing the efficiency of the hardware used. The optimization methods applied in this work are provided on the ground of well-developed FIR filter design methods.

3.3.1 Optimization Techniques Used

The following optimization techniques are used in the proposed FIR filter architecture:

1. The combination of clock gating and ce.
2. Pipelining
3. Symmetric FIR architecture
4. Coefficient symmetry exploitation
5. Pre-adder optimization
6. Multiplier reduction
7. DSP slice optimization
8. Operand isolation
9. Resource sharing
10. Balanced adder tree
11. Parallel accumulation
12. Register-based pipelining
13. Reduced switching activity
14. Glitch reduction
15. Efficient MAC structure
16. Critical path reduction
17. Area optimization
18. Timing optimization
19. Sequential logic optimization
20. Datapath optimization
21. Scalable architecture design

The over-all efficiency of the FPGA implementation is improved with these optimization techniques, along with decreasing hardware complexity, power consumption and timing performance. A symmetric FIR architecture and coefficient symmetry minimize the number of multipliers and the number of DSPs used [7][14]. The simplest methods to increase throughput and decrease propagation delay, which are referred to as pipelining and retiming, respectively, are used [6] [17]. The dynamic power dissipation is minimized by low-power optimization techniques like ce, reduced switching activity, operand isolation, and optimized MAC
structure [3][5][9][10][19]. Resource sharing, DSP optimization and balanced adder tree structures enhance hardware efficiency and FPGA resource utilization [1][16][18]. For higher-order filters, there is additional flexibility in implementation by scalable FIR filter architectures and optimized datapath techniques [12][13][20].

3.3.2 Coefficient Symmetry Exploitation

Symmetry is one of the best optimization methods employed in the design of an FIR filter. In symmetric FIR filters, the coefficients are symmetric with respect to the center tap, which makes combining multiplication operations possible.

Pre-adders are used to sum the input samples of the symmetric coefficients, followed by multiplication. This decreases the number of multipliers and DSP slices needed to implement this [7][14].

The fewer arithmetic operations is a factor in:

1. Lower power consumption,
2. Reduced hardware utilization,
3. Improved timing performance.

3.3.3 Pipelining Technique

To minimize critical path delay and increase the operating frequency, the following approach is implemented: Registers are used to break up long combinational paths into smaller stages.

This technique improves: Timing performance, Throughput, Scalability of higher order filters.

As the number of taps increases, pipelining becomes more and more significant. High throughput FIR filter architectures have been widely used with retiming and pipeline optimizations [6].

3.3.4 Resource Share and DSP Optimization

Explain how to share resources and optimize DSP for performance.

Resource sharing is used to minimize hardware overhead by sharing the arithmetic units for more than one operation. Shared adders and multipliers reduce the usage of FPGA resources.

A DSP slice optimization can be done via:

1. Multiplier reduction,
2. Efficient MAC implementation,
3. Coefficient symmetry exploitation.

Minimizing addition delay in the adders and enhancing the efficiency of the datapath by balanced adder trees is also investigated [1][16].

3.3.5 Low Power Optimization

The proposed architecture includes several low power techniques. Smart switching activities are disabled during inactive operations by using ce signals, thus reducing dynamic power consumption.

To reduce unwanted transitions in the arithmetic units, operand isolation and glitch reduction techniques are used. Optimized multipliers and efficient MAC structures enhance energy efficiency [3][9][10].

Together, these techniques decrease the dynamic power consumption, switching activity, hardware overhead.

3.3.6 Scalable FIR Architecture

The architecture optimization is built for support algorithms ranging from 4-tap FIR to 10-tap FIR filter blocks. This design approach is extended to higher-order filters in just as efficient a manner as for lower-orders.

Such a scalable architecture exhibits both flexibility and adaptability for FPGA-based signal processing applications.

3.3.7 Balance Adder Tree Technique

Using an adder tree optimization is an effective way to enhance accumulation performance while reducing the time it takes to propagate signals. For example, adding multiple operands together in a serial manner can cause delays in arithmetic calculations; however, by using a balanced addition tree, arithmetic performance can be enhanced by minimizing the arithmetic depth of the additional operands.

In this new architecture, a parallel accumulation of additions and balanced adder tree implementation have been created in order to enhance throughput, while also reducing the critical path delays for large tap configurations.

3.3.8 Operand Isolation

To lessen excessive switching activity, operand isolation will isolate arithmetic units from changes in input signals when there is no arithmetic computation required in that section of the data path. This optimization technique reduces the dynamic power consumption and improves the low-power performance of FPGA-compliant FIR filters.

3.3.9 Glitch Reduction

Glitches are unwanted transient switches caused by differences in signal propagation delay in combination circuits. Too many glitches will use up power used in switching and decrease speed performance due to timing issues.

The proposed architecture will reduce the number of glitches generated through the use of pipelining and optimised paths within a data structure. Reducing the number of glitches will improve the stability of all signals and improve power efficiency overall.

3.4 Verilog Implementation

The FIR filters are implemented using Verilog HDL. Separate modules are developed for different tap lengths (4,5 6,7 8,9 up to 10 taps) for better analyzing the FIR Filter

Each module includes:

1. Input signal definitions,
2. Clock and reset signals define,
3. ce logic,
4. Shift registers,
5. Coefficient initialization,
6. Multiply-accumulate operations,
7. Output generation

Testbenches are also developed to test functionality of every design by inputting input signals and monitoring the output reactions corresponding to it.

3.5 Designing of Testbench

Testbenches are developed to perform functional verification for all the architectures of FIR filters.

The testbench generates:

1. Clock signals,
2. Reset conditions,
3. Input stimulus,
4. Output monitoring.

Various methods of input sequences are used for testing filtering behaviour and correct output generation.

3.6 Simulation Using Xilinx Vivado

Xilinx Vivado is used to simulate all the designs. The simulation is done in the functional mode to ensure that the FIR filter output is correct. The output of simulation is the analysis of the waveform of optimized and non-optimized designs. It can also be created using Vivado to create synthesis reports, which contains information regarding:

1. Power consumption
2. Timing (delay)
3. The use of hardware resources (DSPs, LUTs, Registers)

3.7 MATLAB Analysis

The filter design is verified and the frequency response of the filter analyzed using MATLAB. The input signal has several sinusoidal components and the output is monitored to check the performance and working of the filter.

To ensure such that the filter is within the desired specifications, plots like magnitude response and time-domain signals are generated based on the desired taps configuration.

The MATLAB analysis comprises of:

1. Coefficient generation,
2. Frequency response plots,
3. Magnitude response analysis,
4. Time-domain waveform analysis.

The MATLAB generated coefficients are directly applied in Verilog implementation [2].

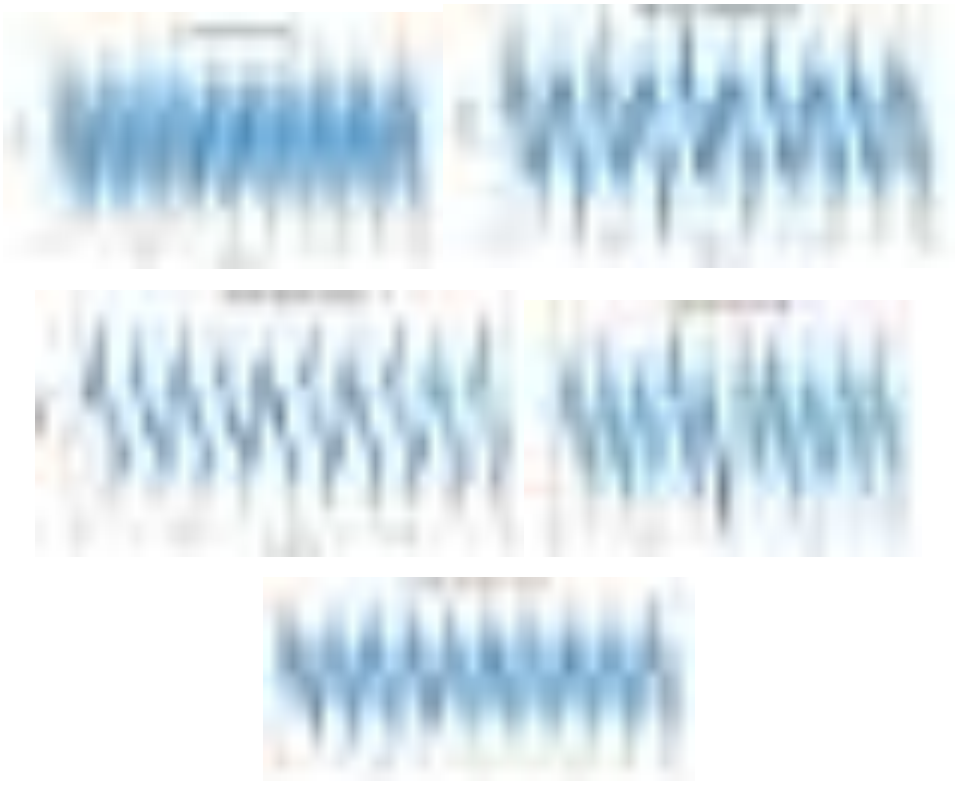


Figure 3.7: MATLAB Plots for LPF, HPF, BPF, BSF

3.8 FPGA Implementation

Proposed architectures are synthesized, and implemented using Xilinx Vivado on FPGA. FPGA implementation is used to analyze:

1. DSP utilization,
2. LUT utilization,
3. Register utilization,
4. Timing performance,
5. Power consumption.

Optimized and non-optimized architectures are compared using the Vivado synthesis reports [13][16][20].

3.9 Performance Parameters

FIR filters performance is measured with the following parameters:

1. Power Consumption: Sum of power consumed by the design.
2. Delay: Time it takes to propagate a signal.
3. Area: Hardware that is consumed (DSPs, LUTs, Registers)

To analyze the improvements, a comparison of the optimized and non-optimized designs is made.

3.10 Summary

This chapter introduced the method and approach of designing and implementing the optimized FIR filters. The proposed architecture was designed with various optimizations including coefficient symmetry exploitation, pipelining, resource sharing, and low-power optimizations. FIR filters were designed in Verilog HDL, simulated with Xilinx Vivado and analyzed with MATLAB.

Chapter 4

Implementation strategy

The implementation strategy used for the development of optimized and non-optimized FIR filter structures from 4-tap to 10-tap are shown in this chapter. The methodology for implementation includes generation of MATLAB coefficients, fixed-point conversion, implementation using Verilog HDL, functional simulation, RTL synthesis and implementation on FPGA using Xilinx Vivado.To further optimize the FPGA implementation efficiency and hardware complexity, several optimization techniques such as coefficient symmetry exploitation, pipeline optimization, low-power optimization with ce, DSP optimization, resource sharing, and balanced adder tree structure are incorporated in the proposed optimized architecture [6][7][14]. The designed FIR filter architectures are simulated and implemented using behavioral simulation and FPGA to ensure proper functionality and performance.

4.1 FIR Filter Design Flow

The implementation process starts with the generation of the FIR filter's coefficients in MATLAB with the tool FDATool. Coefficients of LPF, HPF, BPF and BSF are calculated using MATLAB for the specifications of the required filters. The resulting coefficients are then tested for filter characteristics by plotting frequency response and magnitude response using a plot before the hardware implementation [2].

Once the coefficients are generated, the floating-point coefficients are converted to fixed-point quantized (Q-format) representation. Fixed-point implementation makes it easier to use an FPGA and decreases hardware complexity.

The resulting converted coefficients are implemented in the Verilog HDL (VHDL) of four different FIR filter architectures, ranging from 4-tap to 10-tap. The optimized and non-optimized architectures are separated by Verilog modules.

Testbenches are used in Xilinx Vivado to perform functional verification. Behavioral simulation is performed to check correct performance of the shift registers, multiply-accumulate and output of the filter.

Successful simulation followed by RTL Synthesis and FPGA Implementation with Xilinx Vivado. Various analysis techniques such as timing analysis, power analysis, DSP utilization, FPGA resource utilization are carried out to assess the effectiveness of the applied optimization techniques.



Figure 4.1: Flow Chart of FIR Filter Design Flow

4.2 MATLAB-Based Coefficient Generation

Designing FIR filter, generating filter coefficients and frequency response analysis are done using MATLAB. MATLAB FDATool is an efficient tool to design digital filters and verify filter properties prior to hardware realization [2].

The coefficients of FIR filters are produced for LPF,

HPF, BPF, BSF.

The process of filter design consists of:

1. The order in which the filtering is performed,
2. Determination of cut-off frequency,
3. The frequency at which samples are taken,
4. Window function selection,
5. Frequency response verification.

An analysis of the coefficients generated by the computer is performed in terms of their amplitude and frequency response characteristics prior to hardware implementation.

4.2.1 LPF Coefficient Generation

The coefficients of LPF are calculated with MATLAB, where we enter the desired cutoff frequency and filter order. By using this formula, we can let low-frequency components pass through while attenuating high-frequency ones.

These coefficients are verified using frequency response analysis prior to conversion to fixed-point.



Figure 4.2.1: 4-10 Taps LPF Coefficients

4.2.2 HPF Coefficient Generation

The generation of the HPF coefficients is to suppress lower frequencies and pass higher frequencies through the filter.

The HPF characteristics are verified by using MATLAB frequency response analysis before implementation in FPGA.

Figure 4.2.2: 4-10 Taps HPF Coefficients

4.2.3 BPF Coefficient Generation

These BPF coefficients are used to pass signals in a desired frequency band and suppress signals at frequencies not in the band.

The coefficients obtained are checked using MATLAB magnitude response analysis.



Figure 4.2.3: 4-10 Taps BPF Coefficients

4.2.4 BSF Coefficient Generation

This module generates the coefficients for the BSF. This module generates the coefficients of the BSF.

The BSF coefficients are created to block signals in a specific frequency band while passing signals of other frequencies.

MATLAB analysis is performed to check the filter response prior to implementation on hardware.



Figure 4.2.4: 4-10 Taps BSF Coefficients

4.2.5 Fixed-Point Coefficient Conversion

The MATLAB generated coefficients are first stored in floating point format. Because the implementation of the FPGA is more efficient using fixed-point arithmetic, the coefficients are fixed-point represented by coefficients in Q format [8].

These are the steps involved in the conversion process:

1. Scaling,
2. Quantization,
3. Integer representation.

The converted coefficients are then used for the Verilog HDL implementation.

4.3 Verilog HDL Implementation

To realize the FIR filters in an efficient manner on FPGA, the architectures are coded in Verilog HDL.The FIR filter architectures are coded in Verilog HDL to realize efficiently on FPGA board. FIR filters of various sizes (between 4-tap and 10-tap) are processed in separate modules.

The implementation mainly consists of shift registers, multipliers, adders,

MAC units, output registers.

4.3.1 Implementation of Shift Register

To perform an FIR filtering operation, the input signal requires a delayed version. The shift register executes implementation of delay samples. Shift registers act as a delay line that advances the input data with each clock pulse.

4.3.2 MAC Operations

The multiplication of FIR coefficient and a delayed sample and the accumulation of all such multiplications is Called MAC.

The MAC implementation allows for more efficient arithmetic and FPGA resource usage [9][10].

4.3.3 Architectures of FIR Filter with 4 -10 taps

Separate architecture for 4-tap to 10-tap FIR filter using Modular Verilog HDL.

The architectures that have been carried out support LPF, HPF. BPF

BSF activities.

It has been observed that all types of filters can be implemented efficiently using odd-tap FIR filters. Even tap FIR filters also seem to show an improved suitability mainly for realization of LPF and BPF.

4.3.4 CE based Low-Power Operation

Logic that enables the clock signal is inserted for disabling the switching activities when operations of module are inactive. Disabling CE statement prohibits any mathematical action and registers will not get updated. As a result, dynamic power consumption is reduced [2][3][19].

4.4 Implementation of Non-Optimized FIR filter

The non-optimized FIR design will be carried out using the traditional FIR design technique. Each tap will have its own multiplier and adder. Even though the architecture executes the filtering operations correctly, it generates a high volume of. Hardware complexity has increased in addition to the propagation due to an increase in the use of logical resources on DSP. In order to reference our FIR filter optimized implementation, we will use a non-optimized implementation for comparison.

Figure 4.4: Non-Optimized Verilog Code for 10 Tap

4.5 Optimized FIR Filter Implementation

In the proposed pattern, the FPGA implementation efficiency of the FIR filter is enhanced using various optimization techniques. This refers to hardware complexity and critical path delay. We implement the optimization techniques. Special multipliers are shared in the exploitation of coefficient symmetry to reduce cost. Pipelining high throughput rate The utilization of DSP optimization is to lower the FPGA utilization of resources and take advantage of the timing performances, various optimization techniques [6][7][14].



Figure 4.5: Optimized Verilog Code for 10 Tap

4.6 Verification of Functional Logic and Testbench Design

All the architectures of FIR filters are verified using testbenches. The testbench results include clock signals, resetting conditions, input stimulus ,managing the outputs. Working of designed FIR filters are verified using Xilinx Vivado using behavioral simulation.



Figure 4.6: TestBench For 10 Tap without and with optimized

4.7 RTL Implementation and Hardware FPGA Synthesis

Xilinx Vivado is used to conduct RTL synthesis and FPGA implementation.

The RTL synthesis will generate:

1. RTL diagrams,
2. Reports of the Logic Utilization,
3. DSP Utilization Report Reports on Timing Analysis.

The design that has been synthesized is checked on FPGA.







Figure 4.7: RTL Analysis for 10 Tap with and without optimized and XDC Constraint File for 4-Tap without and with optimized

4.8 Summary

This chapter explained how we built both optimized and non-optimized FIR filter architectures, covering setups from 4-tap up to 10-tap versions. We went through the whole process—using MATLAB to generate coefficients, converting them to fixed-point, writing the Verilog HDL, then running simulations and synthesis, and finally testing everything on an FPGA with Xilinx Vivado. For the optimized FIR designs, we didn't just stick to one approach. We used several tricks to speed things up and make them use fewer resources on the FPGA. That included taking advantage of coefficient symmetry, pipelining, using Clock Enable for lower power, resource sharing, DSP block optimization, and building balanced adder trees. Together, these techniques made the filters more efficient and kept hardware complexity down.

Chapter 5

Results and Discussion

This chapter reports on the study we did of the simulation, synthesis, and FPGA implementation of the put forth FIR filter architectures which range from 4-tap to 10-tap. We go into MATLAB based analysis, functional verification, RTL synthesis, timing analysis, power analysis, and FPGA resource utilization in this chapter. We implemented both the optimized and non-optimized FIR filter architectures out to study the results of applying optimization techniques to hardware performance. We used as our optimizations methods coefficient symmetry play, pipelining, Clock Enable based low power operation, resource sharing, DSP optimization, and balanced adder tree structures [6][7][14][17]. We present that which put forth optimized FIR filter architecture see improvements in FPGA implementation efficiency, timing performance and power consumption.

5.1 Results of MATLAB Simulation

MATLAB's FDATool is used for analysis of the designed FIR filters frequency response which we do before FPGA implementation [2]. We generate and check the FIR filter coefficients for LPF, HPF, BPF, and BSF via magnitude response plots. Once the coefficients are generated they are put into fixed point format and used in Verilog HDL implementation.

5.1.1 LPF Response

The LPF is built to deny high frequencies and allow low frequency components to pass through. Using MATLAB to analyze the LPF's frequency response shows that the LPF is operating correctly with the cutoff frequency that was selected.

The response obtained shows that there is a smooth response in the passband and an attenuated or rolled-off response is found in the stopband.

5.1.2 HPF Response

The High-Pass Filter provides for attenuation of the lower frequency components and passes the higher frequency components.

The MATLAB response verifies that the high-pass filter is functioning properly and is providing sufficient attenuation to the lower frequency components.

5.1.3 BPF Response

The Band Pass Filter permits signals within its frequency range and simultaneously attenuates frequencies that are outside of this range.

With the MATLAB response obtained, both Passband operation and Frequency Selectivity are functioning correctly.

5.1.4 BSF Response

The BSF lets all frequencies through the filter except for some within a specific frequency band, which are rejected.

The confirmation received through MATLAB indicates that the filter has the correct stop band properties and has attenuated those signals that are received.

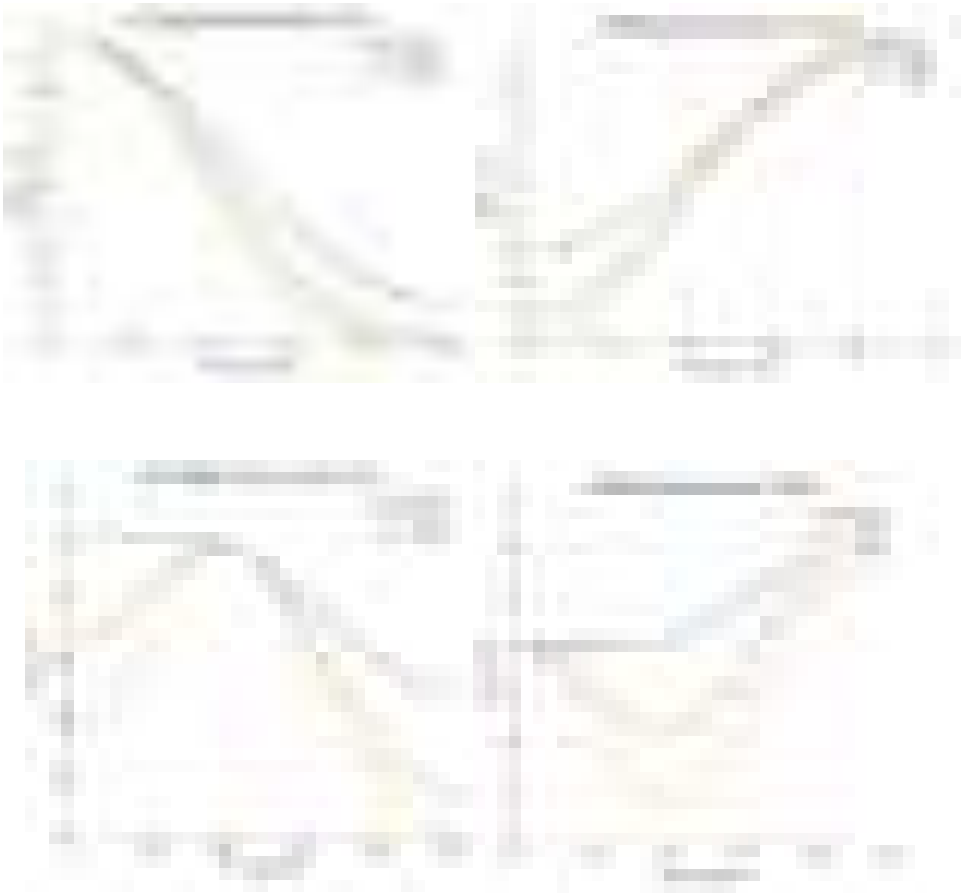


Figure 5.1: Graph Plots of Magnitude vs Frequency for LPF, HPF, BPF, BSF

5.2 Simulations using Xilinx Vivado

Xilinx Vivado is used for executing a behavioral simulation of the FIR filter systems to test design functionality [2][20]. The generation of testbenches has included the generation of all needed clock signals, resets, input stimulus vectors and monitoring of the output.

Simulation Waveforms verify:

1. Operation of the shift registers,
2. All functions of the MAC,
3. Generation of Output,
4. Operation of ce

5.2.1 Simulation of 4-Tap FIR Filter

The behavioral simulation of the 4-tap FIR filter confirms that the shift registers, the multiplication operations, and the accumulation of the products perform as expected.

The output waveform confirms that the FIR filter performs as required of the filter.

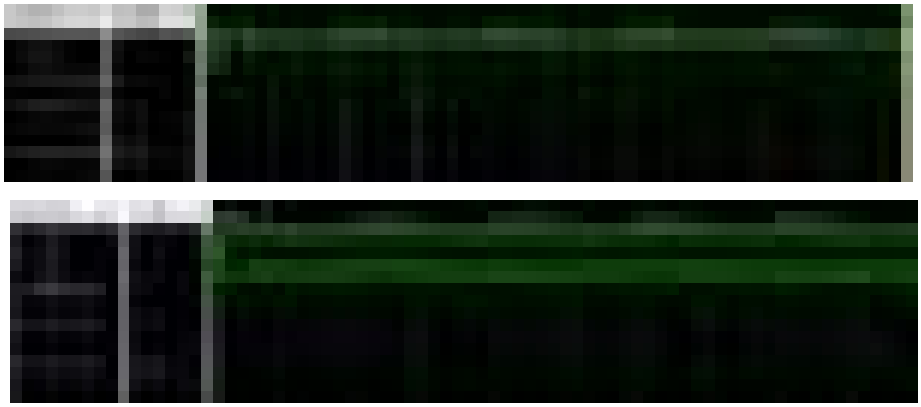


Figure 5.2.1: Without and With Optimized Simulation of 4-Tap FIR

5.2.2 Simulation of 5-Tap FIR Filter

The simulated of the 5-tap FIR filter displays improved characteristics and stable operation of the LPF, HPF, BPF, and BSF.

The displayed waveform confirms the proper operation of the arithmetic operation of the FIR filter.

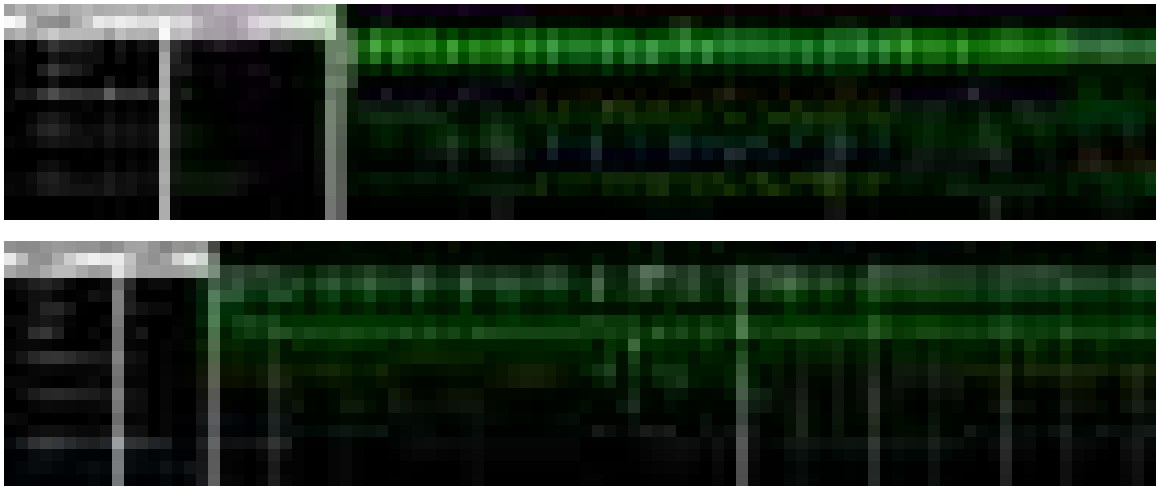


Figure 5.2.2: Without and With Optimized Simulation of 5-Tap FIR

5.2.3 Simulation of 6-Tap FIR Filter

Simulation of the 6-tap FIR filter verifies correct operation of the higher-order datapath structure.

The simulation results verify the correct operation of the MAC unit and the accumulation of the multiplication results.



Figure 5.2.3: Without and With Optimized Simulation of 6-Tap FIR

5.2.4 Simulation of 7-Tap FIR Filter

The 7-tap FIR filter shows the stable operation of the filter with improved frequency selectivity.

The simulation of the waveform verifies the proper operation of the datapath.

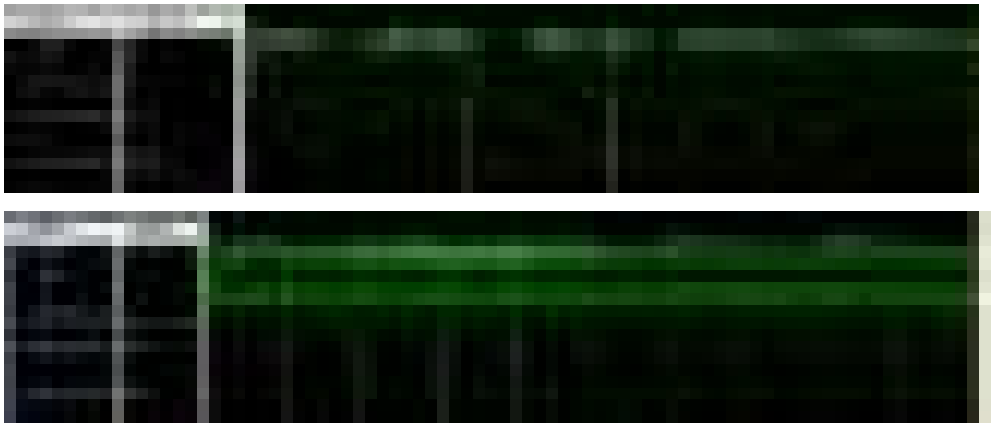


Figure 5.2.4: Without and With Optimized Simulation of 7-Tap FIR

5.2.5 Simulation of 8-Tap FIR Filter

The behavioral simulation of the FIR filter verifies the correct functionality of the pipelined and optimized architecture.

The output waveform indicates the correct performance of the arithmetic operation and output generation.

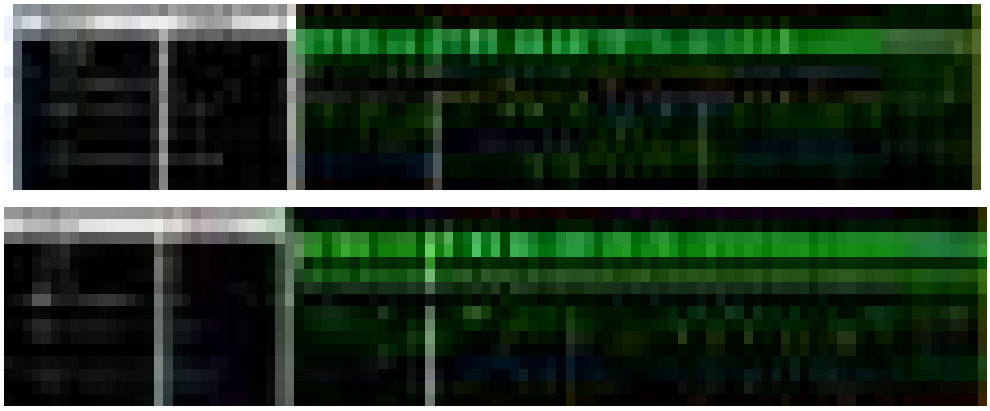


Figure 5.2.5: Without and With Optimized Simulation of 8-Tap FIR

5.2.6 Simulation of 9-Tap FIR Filter

The 9-tap FIR filter exhibits a stable operation for all the considered filter configurations. The obtained simulation results exhibit the improved performance of the filtering process and the correct operation of the optimized structure of the digital filter.

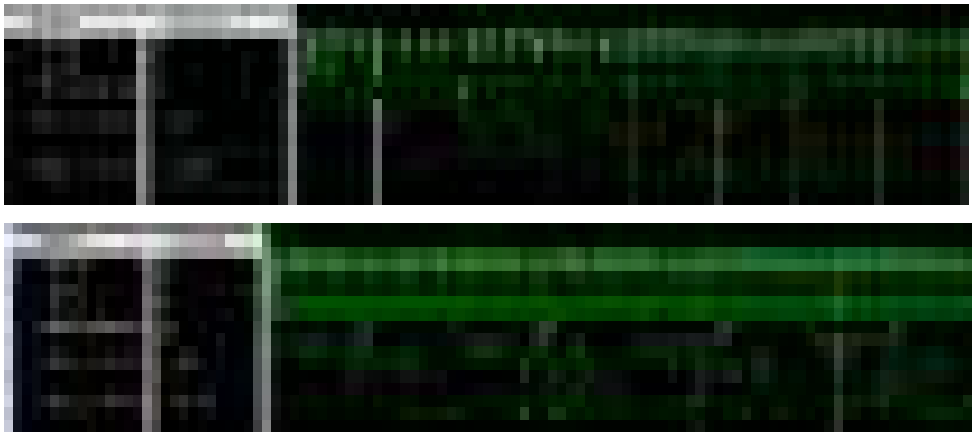


Figure 5.2.6: Without and With Optimized Simulation of 9-Tap FIR

5.2.7 Simulation of 10-Tap FIR Filter

The 10-tap FIR filter represents the highest-order architecture implemented in this work. The simulation results confirm that the datapath and MAC structure has correctly performed its operations.

The displayed waveform confirms the stable operation of the proposed architecture and its filtering function.

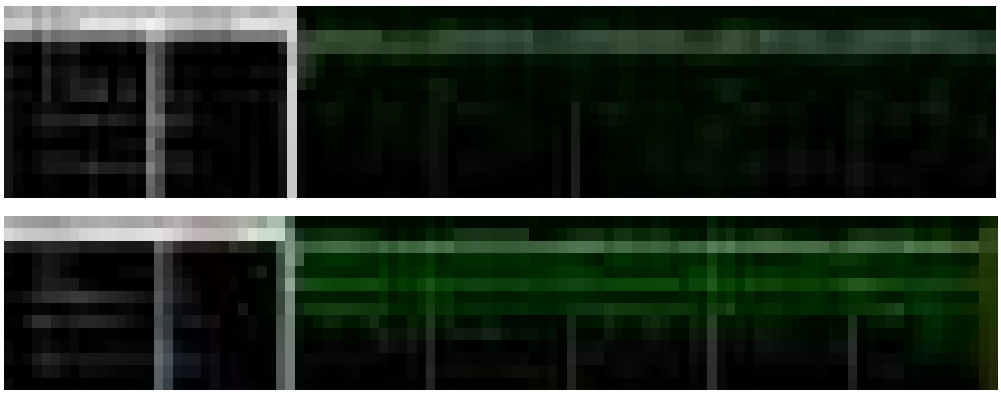


Figure 5.2.7: Without and With Optimized Simulation of 10-Tap FIR

5.3 RTL Synthesis Results

The hardware structure of the FIR filter architectures implemented were analyzed using RTL synthesis with Xilinx Vivado [1][20].

The RTL schematic documentation of the filter calculated interconnections for shift registers, implementations of multipliers, structures of adders, MAC operations, and connections of all output registers.

The architecture that has been optimized shows that there is a greater level of efficiency between hardware and organization within the newly designed architecture than in the non-optimized architecture through reduced arithmetic complexity.

5.4 Hardware Implementation and FPGA Resource Utilization Analysis

Successfully implemented 4 Tap without and with optimized for all modes such as LPF, HPF, BPF, BSF on FPGA Nexys A7-100T board.Xilinx Vivado synthesis reports are used to perform FPGA resource usage analysis. FPGA implementation parameters include LUT, FF and DSP slice utilization, and will be analyzed for optimized vs. non-optimized architectures. Optimized architectures demonstrate greater utilization of FPGA resources as a result of fewer arithmetic operations and more efficient organization of the datapath [1][7][18].



Figure 5.4: FPGA Implementation of 4-Tap without and with Optimized

5.5 Comparison of Non-Optimized and Optimized Architectures

The comparative analysis between optimized and non-optimized multi-mode FIR filter architectures implemented using Verilog HDL (in Xilinx Vivado) is included in this section. Analysis was done for different configurations of taps (4 to 10) and different modes of operation (LPF, HPF, BPF, and BSF).

The optimized architectures utilize several different optimization techniques such as pipelining, coefficient symmetry exploitation, clock enabled (ce) low-power operation, balanced adder tree implementation, resource sharing, and an optimized data path structure.

The results of the comparative analysis include FPGA resource utilization, timing performance, power consumption, and overall hardware efficiency.

5.5.1 Comparison of Resource Utilization

Xilinx Vivado is used to create synthesis reports for FPGA resource utilization analysis. Optimized and non-optimized LUT usage, register usage and DSP slice usage are compared. The optimized architectures show how the coefficient symmetry is exploited to reduce the FPGA resources and how efficient the arithmetic implementation of the architectures is. A number of techniques such as resource sharing and multiplier reduction can be used to reduce the complexity of the hardware in higher order FIR filter architectures. Optimization based on coefficient symmetry makes a significant reduction of the requirements for the multipliers, by using the symmetry properties of the coefficients [14]. The efficient use of the DSP and optimization of the MAC also helps to minimize the number of arithmetic resources used [9] [16].

According to the synthesis reports, optimized architectures need less LUTs and less DSP resources than the conventional non-optimized FIR filter implementations. This improvement is more pronounced as the number of taps is raised from 4 taps to 10 taps.



Figure 5.5.1: Area Analysis of 10 Tap for both non-optimized and optimized

5.5.2 Timing Analysis

Timing analysis is done using Vivado timing reports to compare propagation delay and critical path of the FIR filter architectures. The optimized architectures require less timing since they are pipelined and have balanced accumulation.

To minimize the combinational delay, the following is done in pipelining: Arithmetic operations are broken down into a series of stages. This not only increases maximum operating frequency but also boosts throughput performance. Pipelined FIR architectures are also reported in [6] and [17] which show similar timing improvement.

The optimized architectures have less propagation delay than non-optimized architectures. Pipelining and optimized datapath structures provides better timing stability for higher order FIR filter implementation and achieve critical path reduction.

Balanced adder tree implementation also helps to reduce arithmetic depth and increased efficiency of accumulation in FIR filter's operation.



Figure 5.5.2: Delay Analysis of 10 Tap for both non-optimized and optimized

5.5.3 Analysis of Power

Vivado power reports are generated after synthesis and implementation and used to perform power analysis. The comparison includes dynamic power consumption and overall FPGA power utilization for optimized and non-optimized FIR filter architectures.

The optimized architectures show reduction in power consumption due to ce based operation, operand isolation, glitch reduction, and minimization of switching activity. There is also a reduction in the arithmetic complexity, which further reduces dynamic power consumption.

To minimize switching power, Clock Enable optimization ensures that there are no unnecessary register transitions during periods of inactivity computation. Some similar optimization techniques of low power with optimized arithmetic structures and efficient implementation of the MAC have been reported in the past works [3][5] [9][19].

The findings of the analysis suggest that the optimized FIR filter architectures are more energy efficient than the conventional non-optimized architectures.



Figure 5.5.3: Power Reports for 10 Tap without and with Optimized

5.5.4 Discussion on Comparative Performance

The comparative analysis results presented clearly demonstrate that the optimized FIR filter architectures have a great improvement in terms of implementation efficiency on FPGAs when compared with non-optimized architectures. The pipelining, coefficient symmetry exploitation, DSP optimization, balanced adder tree implementation and low-power optimization techniques enable the overall hardware performance.

Comparative analysis of both even tap and odd tap FIR filter architectures for various filter modes is also proposed in the work. It is concluded that even-tap FIR filter architectures are more suited to LPF and BPF implementations due to smooth frequency response characteristics and symmetry of the filters' coefficients.

But odd-tap FIR filter architectures are found to be stable and efficient for all the filter modes like LPF, HPF, BPF and BSF. The odd-tap FIR filters with the centre coefficient are more frequency-symmetric and more flexible when it comes to filtering.

The optimized architectures illustrate:

1. Reducing LUT utilization,
2. Lowering down the DSP consumption,
3. Reduce propagation delay,
4. Minimizing switching activity,
5. Improving the timing performance,
6. Lower power consumption.

The proposed architectures are suitable for low-power and better performance digital signal processing applications using FPGAs due to these improvements. The same type of FPGA optimization techniques of FIR filter architecture has also been reported in the previous research [6][17][20].

Comparison of optimized and non-optimized FIR filter architectures was conducted based on: FPGA Resource Utilization, Timing Performance, Power Consumption, Hardware Complexity .The optimized architecture has demonstrated improved efficiency in FPGA resource use through reduced arithmetic overhead as a result of implementation-level optimization techniques [6][14][17].

Also, in the table comparision,1 DSP block is equivalent to 385 LUTs which is further converted to LUTs conversion for better analysis of area with respect to taps.

Here, Total LUT corresponds to the summation of LUT from DSP Block Equivalent and LUTs as logic.

Table 5.5: Power, Delay, Area Analysis for 4-10 Taps without and with optimized and Improvement of Power, Delay and Total LUT

Taps	Power WO (W)	Power OPT (W)	Delay WO (ns)	Delay OPT (ns)	LUT WO	LUT OPT	LUT as logic WO	LUT as logic OPT	Total LUT WO (DSP+LUT as logic)	Total LUT OPT (DSP+LUT as logic)	REG WO	REG OPT
4	0.127	0.126	9.007	8.496	1540	770	21	20	1561	790	65	65
5	0.129	0.126	10.838	7.282	1925	1155	25	25	1950	1180	81	113
6	0.130	0.127	12.423	8.722	2310	1155	28	25	2338	1180	97	129
7	0.131	0.129	14.035	7.691	2695	1540	27	42	2722	1582	113	145
8	0.133	0.129	16.056	11.178	3080	1540	27	27	3107	1567	129	129
9	0.135	0.130	20.330	11.029	3080	1925	111	29	3191	1954	176	145
10	0.135	0.129	19.675	10.716	3850	1540	28	57	3878	1597	161	145

Taps	Power Improvement (%)	Delay Improvement (%)	Total LUT Improvement (%)
4	0.79%	5.67%	49.39%
5	2.33%	32.81%	39.49%
6	2.31%	29.79%	49.53%
7	1.53%	45.20%	41.88%
8	3.01%	30.38%	49.57%
9	3.70%	45.75%	38.77%
10	4.44%	45.54%	58.82%

Table 5.5 gives a comparison between non-optimized vs. optimized architectures of FIR filters (4, 6, 8, and 10 taps) based on power consumption, propagation delay, LUT utilization, and register utilization as defined by Xilinx Vivado synthesis reports. Results show that optimization techniques such as pipelining, using symmetry in filter coefficients, using a clock enable based low-power operation, using a balanced adder tree for the implementation of FIR filters, and using resource sharing all lead to significant improvement in the efficiency of the implementation of FIR filters in FPGAs. Furthermore, the results indicate that when optimizing the implementation of FIR filters in FPGA's, both power consumption and propagation delay will be decreased; whereas LUT utilization will also be minimized relative to the implementation of FIR filters using conventional non-optimized methods.

Table indicates the percent gain of optimized FIR filter architectures over non-optimized architectures. Based on the analysis, it can be seen that there is a significant decrease in propagation delay and LUT utilization for all the different tap configuration. The 7-tap, 9-tap and 10-tap architectures achieve a maximum delay improvement of around 45% and an improvement of almost 59% in LUT utilization for the 10-tap optimized architecture. The power consumption reduction is also consistent in all the filter configurations. The results show that the proposed optimization methods are very effective in improving the efficiency of FPGA implementation and also offer scalable low power FIR filter architectures for use in DSP applications.

5.6 Filter Performance Analysis

FIR filter architectures were evaluated using various configurations such as LPF, HPF, BPF, and BSF. Implementation results showed that odd-tap FIR filters are stable and have a successful implementation for all configurations. On the other hand, even-tap FIR filters were determined to be more suitable only for LPF and BPF configuration. The optimised architecture proposed in this project provides the basis for an efficient method of scaling up to realise higher order FIR filter implementations on an FPGA efficiently.

The work under proposal is to compare and contrast the even tap and odd tap FIR filter architectures for LPF, HPF, BPF and BSF implementations. The simulation and synthesis results show that the performance of the filter will change with the different tap configuration and the different filter mode.

It is noted that even-tap FIR filter architectures offer better performance for LPF and BPF implementations since the filter coefficients are more symmetric and have smoother frequency response characteristics. Even-tap structures can effectively eliminate the unwanted frequency components and offer stable filter response for LPF and BPF configurations.

But odd-tap FIR filter architectures exhibit efficient and stable operation in all the filter modes, such as LPF, HPF, BPF and BSF. The coefficient of the center tap of odd number tap FIR filters can be used to improve frequency symmetry and provide greater flexibility in filter design. The symmetric coefficient property of the FIR filter architectures also helps to reduce the number of multipliers and arithmetic complexity [14]. Optimal FIR filter architecture, symmetric coefficient implementation and pipelined datapath structure has been discussed in previous studies [6] and [20].

Based on the analysis in this work, the odd tap architecture is more suitable to implement the generalized multi-mode FIR filter, while the even tap is better for implementing LPF and BPF.

Table 5.6: Various Filters Performance

Tap Type	**LPF**	**HPF**	**BPF**	**BSF**
Even	Good	Moderate	Good	Moderate
Odd	Relatively Good	Good	Relatively Good	Good

5.7 Result Discussions

The we see that which optimization methods we apply very much improves the performance of FPGA implementation of FIR filters. We note that which we see coefficient symmetry is a very effective in reducing multiplications and DSP slice use [7][14]. Also we see that pipelining which is the practice of we use which is to reduce the critical path delay does in fact improve timing performance [6][17]. Also we see that which we may implement Clock Enable based low power operation which what we notice is it reduces switching activity and in turn reduces dynamic power consumption [3][5][19].

We also see that our optimized architecture does very well in terms of scale from 4 tap to 10 tap implementations and at the same time we see it do this which does not sacrifice filtering performance and also we see it does this which also reduces hardware complexity.

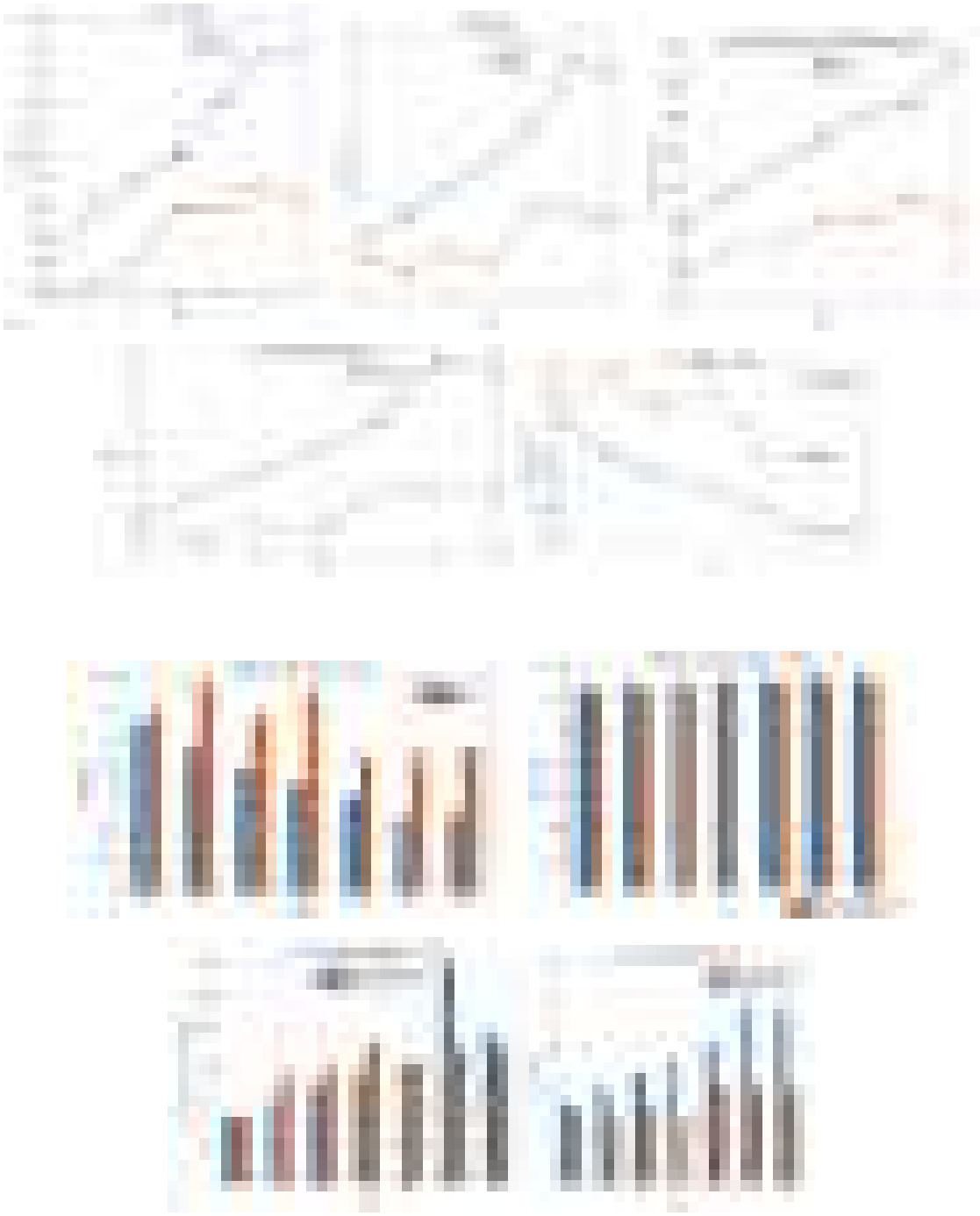


Figure 5.8: Results and Comparison of Power, Area, Delay, Throughput vs Taps

5.8 Summary

The findings of the various optimization stages carried out for designing, verifying, synthesizing, and implementing FIR filters using FPGA technology are outlined here, as well as their performance compared to previous methods

The results show that the proposed optimized architecture provides better overall efficiency with respect to utilizing resources (FPGA), lower power consumption, and less DSP usage and improved overall timing compared to conventional designs.

Chapter 6

Conclusions and Future Scope

6.1 Concluding Remarks

The present-day work includes the design and FPGA implementation of optimization and non-optimization of FIR filter architecture (the 4, 5, 6, 7, 8, 9, and 10 taps FIR filter architecture are being designed) using Verilog HDL and Xilinx Vivado. MATLAB FDATool was utilized to create the FIR filter coefficients for LPF, HPF, BPF, and BSF configurations and fixed-point version for FPGA.

Separate FIR filter architectures were implemented and verified using behavioral simulation, RTL synthesis, timing analysis, and FPGA implementation. The proposed work aims at achieving optimization to enhance efficient FPGA implementation using the following techniques such as exploitation of coefficient symmetry, pipelining, ce based low power operation, resource sharing with balanced adder tree and DSP optimization [6][7][14][17]

The FIR architecture was optimized in which the FPGA resources usage, consumption of power and timing results was better than the non-optimized one. The optimized techniques applied were a success reducing multiplication operations, switch activity and improving the efficiency of the datapath respectively [3][5][19].

Simulation of the behavior verified that all the FIR Filter architectures worked correctly from 4-tap to 10-tap. Using Xilinx Vivado confirmed that RTL synthesis and FPGA implementation successfully realized the hardware of the proposed architectures.

The implementation results also showed odd-tap FIR filters can execute LPF, HPF, BPF, and BSF in a stable and efficient manner. Conversely, even-tap FIR filters were found to be more suitable for LPF and BPF implementations.

The optimized FIR architecture proposed for implementation of higher-order filters is highly scalable with better hardware efficiency and stable filtering performance. Hence, the architecture implemented is suitable for FPGA-based digital signal processing applications that require low power and high performance FIR filtering operation.

6.2 Major Contributions of the Work

The key contributions of the work are presented in this section.

Design and implement verilog HDL-based architectures for 4-tap to 10-tap FIR filters. MATLAB used to generate coefficient and convert floating point data to fixed-point data with same precision for LPF, HPF, BPF and BSF implementations.

Functional verification and implementation of FPGA using vivado.

Optimizing FIR architectures by using:

1. Utilization of Coefficient Symmetry,
2. Pipelining,
3. Low-power functioning based on clock activation,
4. Collaborative Consumption,
5. Optimization of DSPs,
6. Balanced Adder Tree Arrangement.

An elaborate study of optimized and non-optimized FIR filter architectures using:

1. Resource Utilization of FPGA
2. Performance of timing.
3. Energy usage

Examining and assessing filtration performance of even-tap and odd-tap FIR filter architectures.

6.3 Extensions of the Proposed Work

There are several ways that this work could be further refined to increase the performance and scalability of FIR filter implementations which are as follows:

1. Higher-order FIR filters with 10-tap configurations may be utilized in advanced signal processing.
2. Flexible FIR filter architectures can be designed for dynamic filtering.
3. ASIC implementation can be used to evaluate area and power optimization on chip level.
4. Advanced multiplier optimization techniques such as Booth multipliers, Wallace tree multipliers, and approximate multipliers can be used to further reduce power consumption and arithmetic complexity [4][9][10].
5. For automated FIR filter architecture optimization, artificial intelligence and machine learning techniques can be used.
6. Real-time FPGA implementations with high-speed communication and signal acquisition systems could be used for practical DSP applications.
7. Additional low power techniques such as dynamic voltage scaling and advanced clock gating can also be employed to further enhance power efficiency [3][5].
8. The architecture may be used for image processing, biomedical signal processing, wireless communication, and audio processing.

The thesis in this document designs, implements and analyzes the performance of optimized and non-optimized multi-mode FIR filter architectures in MATLAB and Xilinx Vivado. The project proposed was to implement the FIR filters using FPGA in the following modes: 1) LPF), 2) HPF, 3) BPF, 4) BSF using Verilog HDL.

The filter coefficients for different filter configurations were created in MATLAB FDATool and then implemented in Xilinx Vivado for simulation, synthesis and FPGA analysis. FIR filter architectures from 4-tap to 10-tap were designed, analyzed and optimized and non-optimized for both.

To make the FPGA implementation more efficient, several optimization techniques such as pipelining, coefficient symmetry exploitation, Clock Enable based low-power operation, DSP optimization, balanced adder tree implementation, multiplier reduction, resource sharing, operand isolation and glitch reduction were used. These methods have minimized propagation delay, switching activity, hardware complexity and power consumption.

The synthesis and simulation results showed that optimized FIR filter architectures give better performance than the conventional non-optimized architectures. The FPGA resource utilization analysis indicated that optimized datapath structures and the implementation of coefficient symmetry reduced the number of LUTs used in the FPGA, as well as the number of DSP slices used and the complexity of the arithmetic operations performed [14]. Similar improvements in throughput and energy efficiency using pipelining and optimized FIR architectures have also been reported in previous research works [6][17]. The timing analysis showed that pipelined architectures have less critical path delay and higher operating frequency. Power analysis also showed that the dynamic power consumption can be greatly reduced through optimization using the Clock Enable and reduction of switching activity. Earlier works [3] [5] [9] and [19] have discussed similar low power implementation techniques for an FIR filter based on optimized arithmetic structures and MAC (Multiplier Accumulator) implementation.

A comparative analysis of even-tap and odd-tap FIR filter architectures for various modes of the filter was also proposed. Based on the observations, it was found that even-tap architectures yield efficient performance for LPF and BPF implementations due to the better frequency response characteristics and coefficient symmetry of this type of architecture. But an odd-tap FIR filter showed stable operation and better flexibility in all the filters (LPF, HPF, BPF and BSF). Results from the simulations present a general improvement in the efficiency of the FIR filters with the application of the proposed techniques, making them suitable for implementation in the low-power and higher performance DSP applications.

In summary, the suggested optimized FIR filter architectures have been able to enhance the efficiency of FPGA implementation and have proven to be scalable, low-power and hardware-efficient solutions for digital signal processing applications. The work shows that using several optimization methods is crucial to create high performance FIR filter architectures for the modern FPGA based DSP systems.

References

[1] F. F. Daitx, V. S. Rosa, E. Costa, P. Flores, and S. Bampi, “VHDL generation of optimized FIR filters,” in Proc. Int. Conf. Signals, Circuits and Systems (SCS), IEEE, 2008.

[2] R. K. Dwivedi and R. Dwivedi, “FIR filter implementation using MATLAB FDATool and Xilinx Vivado,” International Journal of Engineering Research & Technology (IJERT), vol. 6, no. 10, Oct. 2017.

[3] C. Maheswari, B. Gowthami, and K. Neelima, “Design of energy efficient and low power asynchronous 8×8 multiplier,” Advances in Natural and Applied Sciences, vol. 11, no. 8, pp. 530–536, 2017.

[4] M. Sakthimohan and J. Deny, “An optimistic design of 16-tap FIR filter with Radix-4 Booth multiplier using improved Booth recoding algorithm,” Microprocessors and Microsystems, 2020.

[5] M. Sumalatha, P. V. Naganjaneyulu, and K. S. Prasad, “Low power and low area VLSI implementation of Vedic design FIR filter for ECG signal de-noising,” Microprocessors and Microsystems, vol. 71, p. 102883, 2019.

[6] P. Patali and S. T. Kassim, “High throughput and energy efficient FIR filter architectures using retiming and two level pipelining,” Procedia Computer Science, vol. 171, pp. 617–626, 2020.

[7] K. Y. Khoo, A. Kwentus, and A. N. Willson Jr., “A programmable FIR digital filter using CSD coefficients,” IEEE J. Solid-State Circuits, vol. 31, no. 6, pp. 868–874, Jun. 1996.

[8] D. Li, Y. C. Lim, Y. Lian, and J. Song, “A polynomial-time algorithm for designing FIR filters with power-of-two coefficients,” IEEE Trans. Signal Process., vol. 50, no. 8, pp. 1935–1941, Aug. 2002.

[9] M. Masadeh, O. Hasan, and S. Tahar, “Input-conscious approximate multiply-accumulate (MAC) unit for energy-efficiency,” IEEE Access, vol. 7, pp. 147129–147142, 2019.

[10] P. Lyakhov, M. Valueva, G. Valuev, and N. Nagornov, “A method of increasing digital filter performance based on truncated multiply-accumulate units,” Applied Sciences, vol. 10, no. 24, p. 9052, 2020.

[11] M. I. Akram and A. U. H. Sheikh, “Efficient implementation of tap delay line filter using high speed digital signal processor,” in Proc. 11th Int. Conf. Information Sciences, Signal Processing and their Applications (ISSPA), 2012.

[12] J. Akter and R. I. Mazumdar, “Efficient design and implementation of a 4-tap FIR filter using Braun multiplier and ripple carry adder (RCA),” in Proc. 3rd Int. Conf. Intelligent Systems, Advanced Computing and Communication (ISACC), IEEE, 2025.

[13] K. S. Ashim and P. Mathur, “FPGA based realization of a high-speed 8-tap FIR filter for signal processing applications,” in Proc. IEEE Global Conf. for Advancement in Technology (GCAT), 2023.

[14] A. Kumar, S. Yadav, and N. Purohit, “Exploiting coefficient symmetry in conventional polyphase FIR filters,” IEEE Access, vol. 7, pp. 162883–162897, 2019.

[15] M. Alawad and M. Lin, “Memory-efficient probabilistic 2-D finite impulse response (FIR) filter,” IEEE Trans. Multi-Scale Comput. Syst., vol. 4, no. 1, pp. 69–82, Jan.–Mar. 2018.

[16] A. M. George, S. K. John, and S. Kala, “FPGA implementation of an efficient FIR filter using double MAC unit,” in Proc. IEEE Int. Symp. Smart Electronic Systems (iSES), 2024.

[17] B. Kishor and K. Raj, “Design and implementation of high throughput and efficient FIR filter architectures using parallel processing,” in Proc. Int. Conf. Recent Innovation in Smart and Sustainable Technology (ICRISST), 2024.

[18] B. R. Rao et al., “Design and implementation of 6-tap FIR filter using MAC for low power applications,” International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 10, no. VI, Jun. 2022.

[19] B. Rashidi, B. Rashidi, and M. Pourormazd, “Design and implementation of low power digital FIR filter based on low power multipliers and adders on Xilinx FPGA,” Univ. of Tabriz, Technical Report.

[20] G. Singh and N. R. Prakash, “FPGA implementation of higher order FIR filter,” International Journal of Electrical and Computer Engineering (IJECE), vol. 7, no. 4, Aug. 2017.













